Class: Algorithms & Data Structures Semester: Spring 2025

Noah Wiley
noahwiley@mail.fresnostate.edu
Lab Number - Section 09

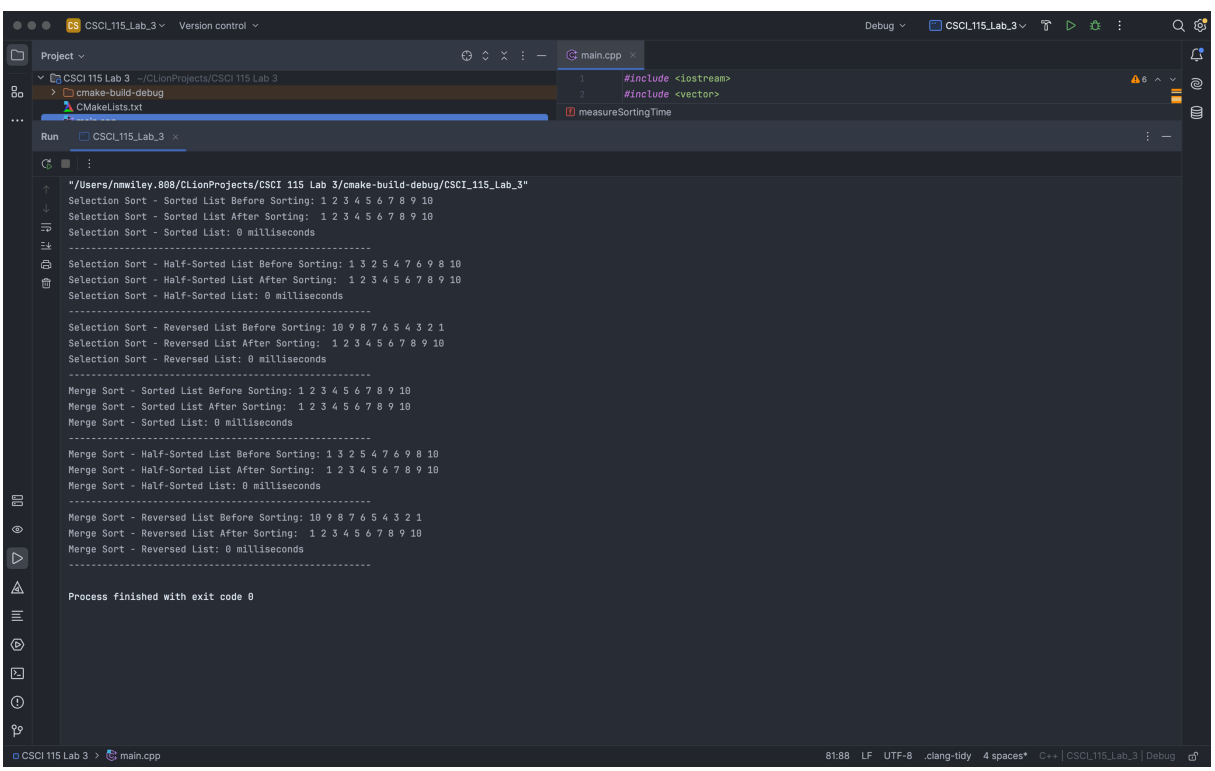
**CSCI 115 Lab 3 Report; Selection & Merge Sort**

- Statement of Objectives
  - The objective of this lab is to gain hands-on experience with two fundamental sorting algorithms: Selection Sort and Merge Sort. The experiment involves implementing these algorithms and analyzing their execution times under three different conditions:
    - When the list is already sorted
    - When the list is half-sorted
    - When the list is completely reversed
  - By comparing the execution times for each case, will evaluate the time complexities of Selection Sort and Merge Sort and discuss their efficiency.

- Experimental Procedure
  - The experiment was conducted using C++ with the following steps:
    - Implementation of Selection Sort:
      - The algorithm iterates through the list, selecting the smallest element and placing it in its correct position iteratively
    - Implementation of Merge Sort:
      - This recursive sorting algorithm follows the divide-and-conquer approach, splitting the array into smaller subarrays, sorting them, and merging them back together
    - Measuring Execution Time:
      - The execution time of each sorting algorithm was measured using the chrono library in C++.
      - Sorting was performed on three types of lists:
        - Sorted List: $\{1,2,3,4,5,6,7,8,9,10\}$
        - Half-Sorted: $\{1,3,2,5,4,7,6,9,8,10\}$
        - Reversed List: $\{10,9,8,7,6,5,4,3,2,1\}$
    - Comparison of Execution Times:
      - The sorting times for each case were recorded and analyzed.

.

- Analysis
  - Interpretation of Results:
    - Selection Sort:
      - The algorithm performed in constant time (0 ms) in all three cases. This might indicate that the sorting operation was optimized away or the list size was too small for a noticeable difference.
    - Merge Sort
      - The sorting times were relatively small, with the reversed list taking the least time (3 ms). The minor variations suggest that Merge Sort is efficient and scales better than Selection Sort for larger lists.
    - Time Complexity Comparison:
      - Selection Sort: O(n^2) in worst, best, and average cases
      - Merge Sort: O(n log n) in all cases




- Encountered Problems
  - Minimal Execution Time Readings:
    - The sorting time for Selection Sort were 0 ms, possibly due to compiler optimizations or the small input size
    - Potential solution: Run the experiment with a larger dataset to observe more notice differences
  - Understanding Merge Sort Recursion:
    - Implementing recursive Merge Sort required careful debugging to ensure proper merging o subarrays

- Conclusions
  - Selection Sort is inefficient for large datasets due to its O(n^2) complexity
  - Merge Sort is significantly faster and scales better with increasing input sizes
  - The experiments demonstrate the impact of initial list order on sorting performance, particularly for Merge Sort
  - Future improvements could include testing on larger datasets and analyzing additional sorting algorithms for further comparison

- References
  - Lab 3 - Selection & Merge Slides